Title

Code Layout, Readability and Reusability – SB Stocks

Code Organization

SB Stocks follows a modular client-server architecture.

The project contains:

- Client folder.
- Server folder.

Backend Code Layout

The backend follows MVC architecture.

Config

Contains database configuration.

Models

Contains Mongoose database schemas.

Controllers

Contains application business logic.

Routes

Contains REST API route definitions.

Middleware

Contains authentication and authorization logic.

Frontend Code Layout

Components

Reusable interface components are stored in the components folder.

Examples include:

- Navbar.
- Login.
- Register.
- Axios configuration.

Context

Shared application state is managed through React Context API.

Pages

Application pages are separated based on functionality.

Examples:

- Landing.
- Dashboard.
- Portfolio.
- History.
- Profile.
- Admin.

Readability

The application uses:

- Meaningful variable names.
- Descriptive function names.
- Modular functions.
- Consistent code formatting.
- Async/await for asynchronous operations.
- Structured error handling.

Reusability

Reusable code includes:

- Axios configuration.
- Authentication middleware.
- Context state.
- Navigation components.
- Database models.
- Stock data services.

Separation of Concerns

Database operations are managed using models.

Business logic is implemented in controllers.

API endpoints are defined in routes.

User interface logic is maintained in React components.

Conclusion

The modular project structure improves readability, reusability, maintainability, and future scalability.